

Resilix

Decentralized control infrastructure for Canton applications

A reusable layer that gives any institutional Canton application decentralized authority, multi-party approval, operator resilience, and a complete audit trail — enforced by the ledger, not by a dashboard.

Executive summary

Canton makes the *ledger* decentralized: settlement is distributed, privacy is sub-transactional, and no single operator sees the whole network. But the **authority over an application running on Canton** — who may move the treasury, who may mint the token, who may pause the protocol — is almost always still centralized. In practice it collapses to one signer, one operator, or one server holding one key. The ledger is decentralized; the control over it is not.

Resilix closes that gap. It is a reusable control layer that any Canton application can adopt to make its privileged actions governed by a quorum of parties, resilient to the failure of individual hosting operators, and fully auditable — with all four properties enforced inside Daml contracts rather than displayed by a user interface. An institutional treasury is the reference application shipped with the project, but Resilix is deliberately not a treasury product. The same contracts protect token administration, trading administration, and protocol governance without modification.

The result: institutional applications on Canton gain decentralized authority and survive the loss of an operator, with every decision provable on-ledger.

The problem

Every institutional application has privileged actions — the small set of operations that move value, change economics, or alter the rules. On Canton today these actions are typically guarded in one of two unsatisfying ways:

1. Centralized administration.

A single admin account, operator, or backend service holds the authority to execute. This is fast to build and easy to reason about, but it reintroduces exactly the single point of control that a decentralized ledger was meant to eliminate. If that key is compromised, that operator is coerced, or that server is down, the application's most sensitive operations are either exposed or frozen.

2. Bespoke approval systems.

Teams that recognize the risk build their own multi-signature or approval workflow — one application at a time. Each is a custom contract set, a custom UI, and a custom audit scheme, re-implemented and re-audited for every product. The logic is nearly identical across them, but the cost and the attack surface are paid again each time.

Both approaches share a deeper flaw: they conflate **who is allowed to act** with **whether the application is**

reachable to act at all. A quorum can be perfectly satisfied and the action still cannot execute because the single operator hosting it has gone offline. Conversely, an application can be highly available and still have no real control over who authorizes its privileged actions. These are two independent properties, and institutional applications need both.

Why it matters for institutional applications on Canton

Canton's design thesis is that regulated institutions can transact on shared infrastructure without surrendering privacy or control. That thesis attracts exactly the applications where a single point of control is unacceptable: tokenized securities, settlement rails, custody platforms, treasuries holding real balances, and market infrastructure whose fee and pause controls move money.

For these institutions, the governing requirement is rarely "can this transaction settle?" It is "can we prove that no one person could have authorized this, and that the system kept running when a participant failed?" That is a question about **control**, not settlement — and it is the question Canton applications currently answer with off-ledger process, spreadsheets of approvers, and operational trust in whoever holds the keys.

An institution evaluating Canton for a treasury or a token program has to satisfy auditors, risk committees, and regulators that:

- No single individual or operator can unilaterally execute a privileged action.
- The failure or compromise of one hosting operator does not take the application down, and does not silently hand control to whoever remains.
- Every authorization decision — who requested, who approved, under what quorum, with how many operators online — is recorded immutably and can be reconstructed later.

Today each institution builds that assurance itself, bespoke, per application. Resilix makes it a shared, verifiable primitive of the platform. That lowers the bar for any institution to deploy a serious application on Canton, which is directly in the interest of the network itself.

How Resilix works

The authorization model

Resilix's central design decision is that authority is expressed and enforced by Daml's own authorization rules, not by application code sitting in front of the ledger. In Daml, a *choice* can only be exercised by the party named in its `controller` clause; the ledger itself rejects any attempt by any other party. Resilix builds its entire control model on this foundation, so there is no privileged path around it:

- A
 privileged action
 is opened against a

Policy

by a party who must be a policy member — enforced by the controller clause and an explicit membership assertion.

- Each

approval

is a separate choice controlled by the approving party. A server cannot manufacture an approval on a member's behalf, because it is not that party, and the ledger will not let it act as one. Duplicate approvals are rejected.

- Execution

is gated inside the

Execute

choice itself: it refuses unless the number of approvals meets the configured threshold

and

the application is adequately hosted. A console that drew the button as enabled would change nothing — the ledger refuses the command.

- Hosting status

is reported by each operator about

its own

node, through a choice only that operator controls. No administrator asserts a node's health on its behalf, and a non-operator cannot even see the hosting contract, let alone report on it.

Because a Daml party's right to act comes entirely from these controller clauses, a client that lied about who it was could not widen its own authority. That property — that authority cannot be forged, only exercised by the party that holds it — is precisely what the control layer exists to guarantee.

The four primitives

Resilix is organized around four composable, application-agnostic primitives:

- Policy

— defines, per application, who may act and how many of them must approve. A policy is only well-formed with a non-empty, duplicate-free member set and a threshold that at least one and at most all members can meet.

- Governance (shared control)

— a privileged action does not execute until a configured quorum of parties has approved it. This is *k-of-n*

approval, enforced on-ledger. If the quorum is not met, execution is refused by the contract.

- Resilience (distributed hosting)

— a set of hosting operators run the application, with a minimum number that must stay online. Each operator self-reports its node status. Taking one offline leaves the application available while the hosting threshold still permits operation; dropping below the threshold blocks execution — again, enforced by the contract, not the UI.

- Audit

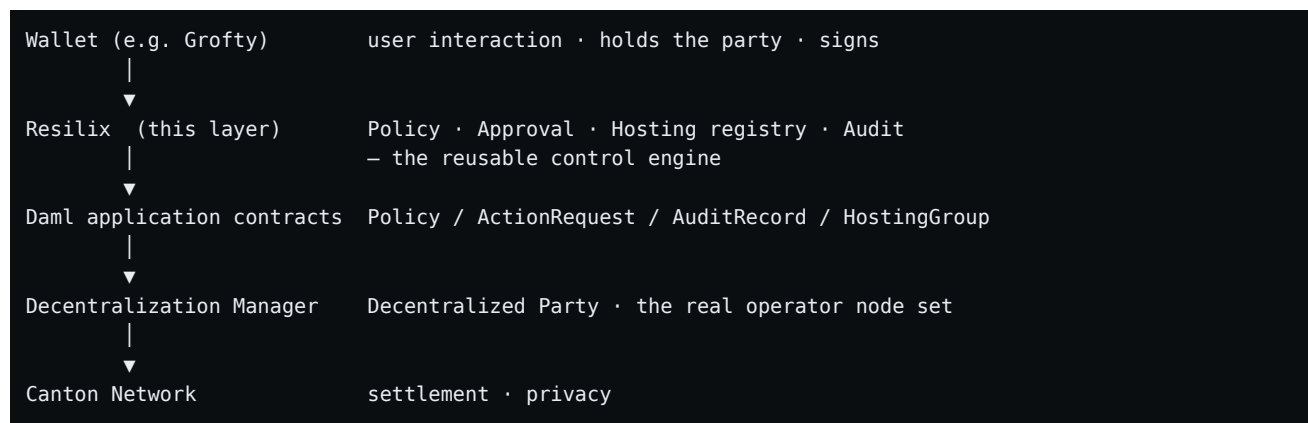
— every request, approval, hosting change, and execution is captured. The execution record even captures how many operators were online at the moment of execution, so the resilience fact is part of the permanent record rather than a transient UI annotation.

Crucially, **governance and resilience are independent guarantees**. Approvals govern *whether an action is*

authorized; hosting governs *whether the application is reachable to execute it at all*. An institution needs both, and Resilix enforces both separately and provably.

Architecture

Resilix occupies a clean position in the Canton stack. It does not reimplement the pieces below it; it depends on them through well-defined interfaces.



Resilix owns application policy and workflow. The user's authority enters through a wallet implementing the published Canton dApp SDK standard (CIP-0103): the application prepares the exact command it would otherwise submit – unsubmitted – and the wallet signs and submits it under the user's own party, so the ledger sees the user's authority rather than a server's claim to it. Beneath Resilix, the Decentralized Party and the real distributed node topology are provided by the network's decentralization infrastructure. The boundary between Resilix and that infrastructure is a contract, not a custom integration – which is what makes the layer portable across deployments.

Use cases

Because the protected action is generic – a *verb*, a *target*, and a *detail* – the same control layer serves a wide range of institutional applications. Four representative instances ship as reference configurations, each with its own membership and quorum:

Application	Privileged action	Quorum	Why it needs Resilix
Treasury	Move 50,000 CC	2 of 3	No single treasurer should be able to move institutional funds; the operation must survive an operator outage.
Token administration	Mint 1,000,000 units	3 of 4	Supply changes are the highest-trust action in a token program; they demand a strong quorum and a permanent record.
Trading administration	Change fee 10 ¢ 25 bps	2 of 3	Economic parameters move real money; changes need shared authorization and an auditable justification.
Protocol governance	Pause the application	3 of 5	An emergency control that must be neither unilaterally triggerable nor single-operator dependent.

The same shape extends naturally to the domains Canton is built for:

- **Treasury management.**
Corporate and institutional treasuries holding on-ledger balances gain enforced dual-control and multi-approver policies, with resilience against the loss of any single hosting operator.
- **Custody.**
A custody platform can place client-asset movements behind a quorum of custodian parties and a hosting group spanning independent operators, so neither a rogue insider nor a single node failure can move or freeze assets. (Resilix governs *who may authorize* the movement; a custody vault that holds the balance is a complementary product the layer would then protect — see *Scope* .)
- **Tokenization.**
Issuers of tokenized securities, funds, or real-world assets place mint, burn, and transfer-restriction controls under shared authority with an immutable audit trail — the exact assurance a securities regulator or fund administrator expects.
- **Financial market infrastructure.**
Exchanges, settlement venues, and protocol operators guard fee changes, parameter updates, and emergency pauses, so no single operator can alter market economics or halt the venue alone.

Scope

Resilix governs *who may act and whether the application is reachable to act* — it is deliberately not a custody or balance-holding product. The reference treasury action is a *request the policy authorizes*, not a transfer out of an escrowed balance, so there is no account balance to hold or display. This is a boundary, not an omission: adding custody would mean a second product (a vault), which this control layer would then protect. Keeping that boundary clean is what makes Resilix reusable across every application above rather than fused to one of them.

What makes Resilix different

Versus centralized administration

Centralized administration — one admin, one operator, one key — is the default because it is the path of least resistance, not because anyone considers it safe. Resilix replaces that single point of control with two independent, ledger-enforced guarantees:

- **No unilateral action.**
Authority is distributed across a quorum of parties, enforced by Daml controller clauses that no server can bypass. Compromising one signer is not enough to execute.
- **No single point of failure.**
Hosting is distributed across operators that self-report their status, and the application remains available as long as the threshold is met. The failure of one operator neither takes the application down nor silently concentrates control.

Where centralized administration offers speed at the cost of trust, Resilix offers the same operational flow with the trust assumption removed — and does so without the application team having to design the guarantee themselves.

Versus custom-built approval systems

Many institutions already know they need multi-party approval and build it per application. Resilix differs in three ways that matter at institutional scale:

- **Reusable, not bespoke.**

One generic contract set protects every application. A new application is a *configuration*

— members, a threshold, a hosting group — not a new contract to write, audit, and maintain. The security review is done once, not once per product.

- **Resilience is a first-class primitive, not an afterthought.**

Most hand-rolled approval systems govern *who approves*

but assume the application is always reachable. Resilix treats hosting availability as an independent, on-ledger guarantee, so an institution gets failure-survival in the same layer that gives it shared control.

- **Enforced and provable, not merely implemented.**

Because every pillar lives in Daml choices, the guarantees are enforced by the ledger and can be independently verified against a live Canton participant. The project includes an automated suite that drives the full request → approve → under-host → recover → execute flow against a real ledger and asserts that execution is refused on-ledger while under-hosted. A bespoke system is only as trustworthy as its last manual review; Resilix's guarantees fail a build when they break.

Who would use it

- **Institutions deploying serious applications on Canton**

— asset managers, banks, custodians, token issuers, and market operators — who need provable control and resilience to satisfy internal risk and external regulators.

- **Application teams building on Canton**

who would otherwise spend significant effort re-implementing approval and resilience logic, and who would rather adopt a reviewed, reusable layer and spend their effort on their actual product.

- **The Canton ecosystem and its decentralization infrastructure providers**

, for whom a drop-in control layer lowers the barrier to bringing institution-grade applications onto the network — which grows the network itself.

Potential business model

Resilix sits in the classic position of shared infrastructure that is costly to build well and valuable to many: a strong fit for a model that monetizes the control plane without taxing settlement.

- **Open core.**
The reusable Daml contract set and reference applications are open and auditable — essential for a security layer, where trust depends on inspection. Adoption is frictionless.
- **Commercial control-plane services.**
Revenue comes from the operational layer institutions actually pay for: hosted policy management and operator onboarding, enterprise audit retention and reporting, compliance integrations, SLA-backed support, and tooling to configure and monitor policies across a fleet of applications.
- **Deployment and integration.**
Professional services to stand up Resilix against an institution's own Decentralized Party and operator set, and to integrate it with existing custody and treasury systems.

The economic logic mirrors the security review itself: an institution pays once for assurance it would otherwise rebuild and re-audit for every application, forever.

How it could grow

1. **Reference layer → production layer.**
Move from the self-contained reference deployment onto a real Decentralized Party on DevNet and then MainNet, turning the modelled operator set into a genuinely distributed one with no change to application logic.
 2. **Catalog of protected applications.**
Expand the set of first-class reference applications (treasury, custody, tokenization, market infrastructure, protocol governance) into a library institutions can adopt and parameterize.
 3. **Policy ecosystem.**
Richer policy types — role-weighted approvals, time-locks, tiered thresholds by action value, delegated and emergency-override policies — all expressed in the same enforced, auditable model.
 4. **Platform integrations.**
Wallet, custody, and compliance integrations that make Resilix the default control plane for new Canton applications rather than a bespoke build each team repeats.
-

Technical approach and value

Resilix's technical bet is that the right place to enforce institutional control is the ledger's own authorization model, and that the right abstraction is a single generic contract set rather than one product. That bet pays off in several concrete ways:

- **Enforcement, not display.**
Policy, approval, hosting, and audit all live in Daml choices. The availability gate is inside the execution choice; it re-checks that the hosting group it is handed is the one for this application, so the gate cannot be bypassed by passing a stale or foreign contract. The guarantees hold even if every line of the user interface is wrong or malicious.
- **Verifiability.**
The full flow is exercised against a live Canton 3.x participant over the Daml JSON Ledger API v2, with

on-ledger checks asserting the invariants — including that execution is rejected while under-hosted and permitted again after recovery — and reproduced from a clean environment in continuous integration. The claims are testable, not just assertable.

- **User authority in the flow.**

Privileged actions are authorized by the user's own Canton party via the published dApp SDK standard, not by a server asserting it is that party. The connection, approval, and execution all carry real authority.

- **Honest boundaries.**

The layer does not reimplement the Decentralized Party or the real multi-participant node topology; it models the

status

of the operator set and enforces the threshold, and treats the underlying distributed deployment as a documented dependency. That discipline is what keeps the layer portable and its claims truthful.

The value, stated plainly: an institution adopting Resilix gets decentralized authority and operator-failure survival for its privileged actions, enforced by the ledger and provable to an auditor, without designing or maintaining that machinery itself.

Future direction

The nearest milestone turns the reference layer into production infrastructure: deploying against a real Decentralized Party on DevNet and MainNet, which is what converts the modelled hosting set into a genuinely distributed one. From there, the roadmap is a broadening library of reference applications, a richer policy model (weighted roles, time-locks, value-tiered thresholds, emergency overrides), and the integrations that make Resilix the standard control plane institutions reach for when they build on Canton.

The destination is simple: when a team decides to put a privileged action on Canton, the question "how do we make sure no one person controls this, and that it survives an operator failure?" should have a reusable, provable answer — and that answer should be Resilix.

In one line

Canton applications get decentralized authority and survive an operator failure — enforced by the ledger, proven on a live one, and reusable by any application.